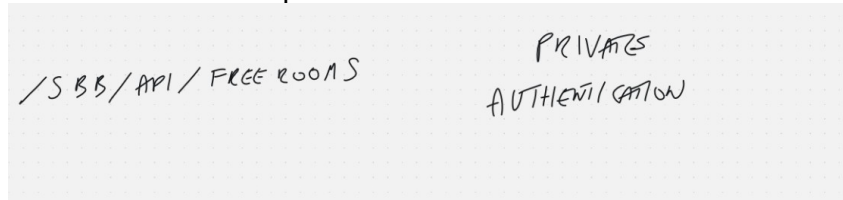


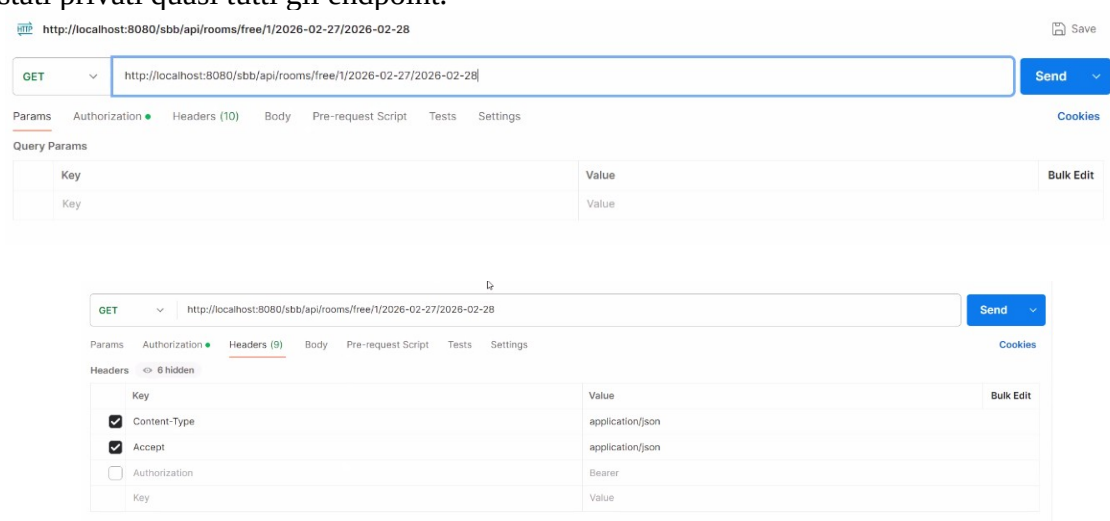
Appunti 02/03/26 – Sicurezza – Brutta copia

Una web application non è disponibile a tutti allo stesso modo. Alcune parti di una web app sono riservate a chi ha determinati ruoli.

Un endpoint è un indirizzo che risponde ad una chiamata HTTP, un indirizzo accessibile dall'esterno. Una API è composta da tanti endpoint. La sicurezza in una web app moderna si traduce in decidere chi può accedere alle api. La sicurezza vera è nel backend. Alcune API sono accessibili, altre no e altre sono accessibili solo a qualcuno.



Sono stati privati quasi tutti gli endpoint.



Gli Headers.

Una richiesta HTTP è composta da una testa e un corpo. La testa contiene gli headers (le intestazioni, che servono per interpretare meglio la request, per aggiungere informazioni corollarie, dettagli alla request, e il body è il contenuto della richiesta (molto spesso è un json).

ContentType indica il tipo del contenuto del corpo.

Accept indica il tipo della risposta

Key		Value
☒ Content-Type	TIPO CORPO	application/json
☒ Accept	TIPO RISPOSTA	application/json
☐ Authorization		Bearer
Key		Value

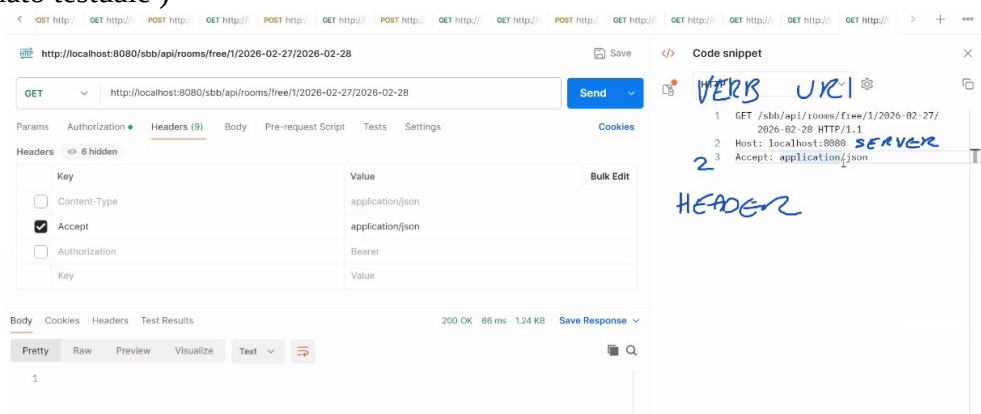
```
1 GET /sbb/api/rooms/free/1/2026-02-27/  
  2026-02-28 HTTP/1.1  
2 Host: localhost:8080  
3 Content-Type: application/json  
4 Accept: application/json
```



REQUEST

In questo caso il corpo non c'è, tutto quello che vediamo è nel header. HTTP 1.1 è la versione della chiamata.

(Approfondimento **HTTP/1.1** (1997): la versione più longeva, ancora largamente usata oggi, che introduce connessioni persistenti, i metodi PUT, PATCH, DELETE e OPTIONS, e lo scambio di dati in formato testuale)

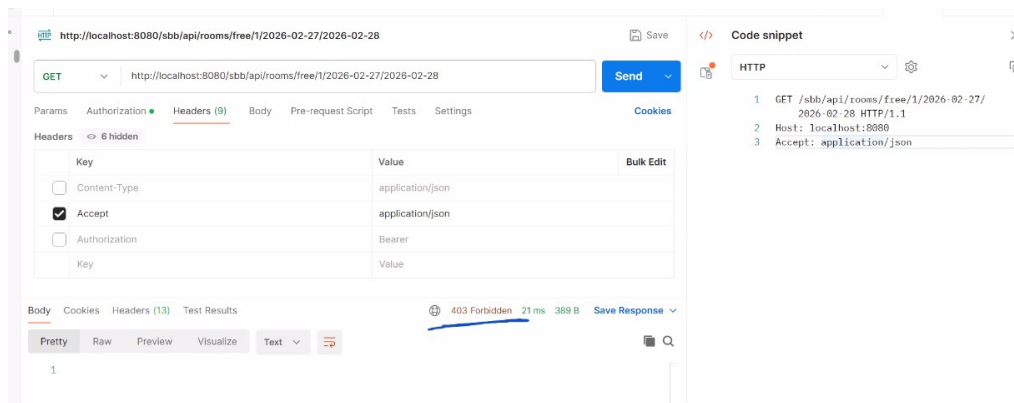


gli **header** sono le informazioni aggiuntive che il client invia al server per aiutarlo a gestire correttamente la richiesta. Non fanno parte del corpo della richiesta, ma forniscono contesto essenziale su come interpretarla e rispondervi. **Accept: application/json** comunica al server in quale formato il client si aspetta di ricevere la risposta, in questo caso JSON

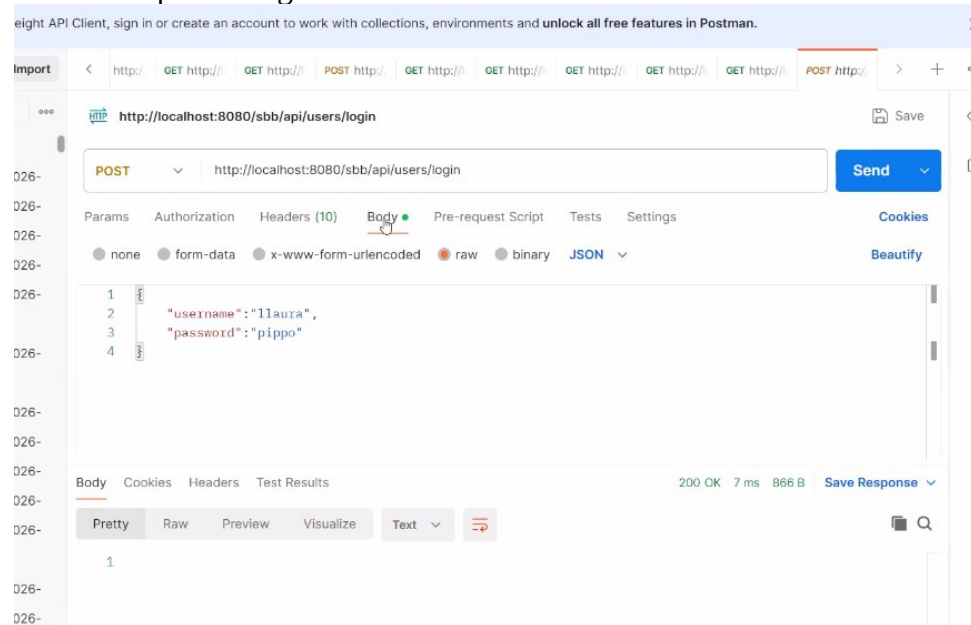
- **Content-Type: application/json** (disabilitato in questo caso) indicherebbe il formato del corpo della richiesta, rilevante soprattutto nelle chiamate POST e PUT

Insieme al verbo HTTP (GET) e all'URL, questi header costituiscono le tre componenti principali di una richiesta HTTP, come annotato nell'immagine stessa con le etichette **VERB**, **URL** e **HEADER**.

Il **client** (in questo caso Postman) costruisce e invia la **request** verso il **server**, che è in ascolto all'indirizzo `localhost:8080`. Il server elabora la richiesta e restituisce al client una **response**, visibile nella sezione inferiore della schermata con il body della risposta, lo status code `200 OK` e i tempi di risposta (66 ms, 1.24 KB).

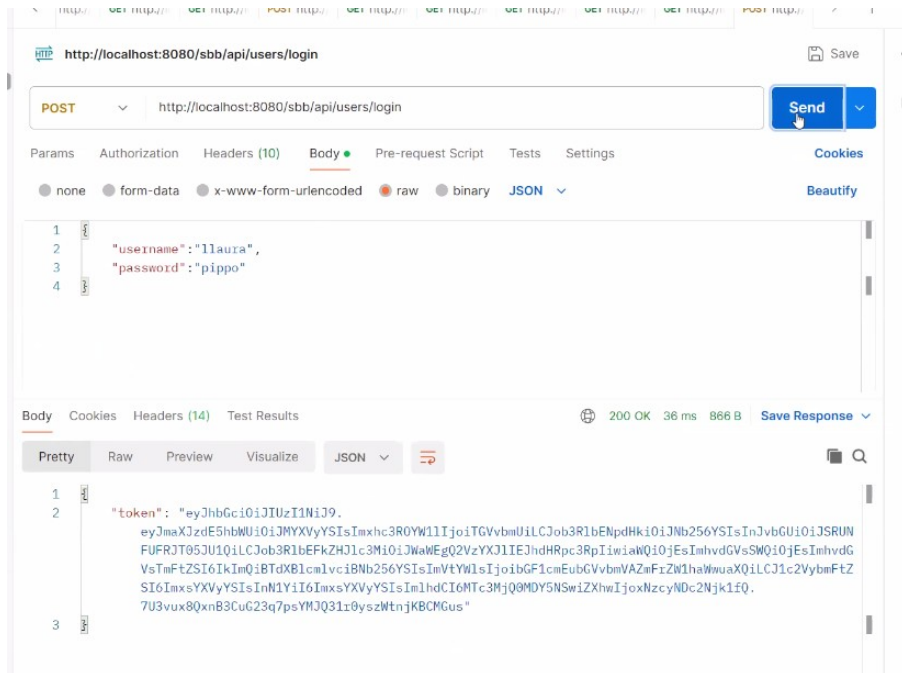


Questa api è ora privata. La prossima volta che qualcuno vorrà contattare la request, contestualmente alla request bisogna autenticarsi.

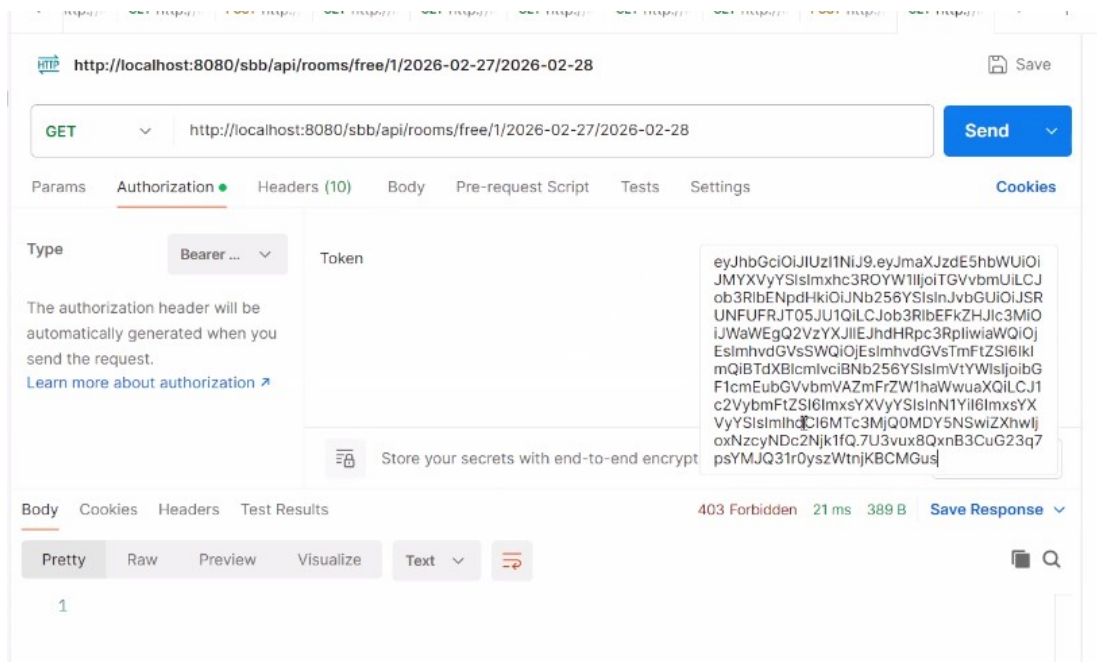


Prima di poter chiamare quel GET dovrò fare una POST e chiamare il login. Se mandassi email e password con ogni request non sarebbe ottimo per le prestazioni per quanto riguarda il db. Il server non vuole ricordarsi di tutti i tuoi client, è il client che ci deve pensare. Il flusso di autenticazione tramite **JWT (JSON Web Token)**. Il client invia al server una richiesta di **login** con le credenziali dell'utente nel body, in questo caso username: "llauro" e password: "pippo". Il server verifica le credenziali e, in caso di successo (risposta 200 OK), restituisce al client un **JWT**, ovvero un token firmato contenente i dati dell'utente. Da quel momento in poi, il client dovrà allegare questo token nell'header **Authorization** di ogni richiesta successiva che richiede autenticazione, esattamente come visto nell'esempio precedente con il **403 Forbidden**: senza il token Bearer il server rifiuta l'accesso. Il JWT permette quindi al server di riconoscere l'utente senza che questi debba reinserire le credenziali ad ogni chiamata.

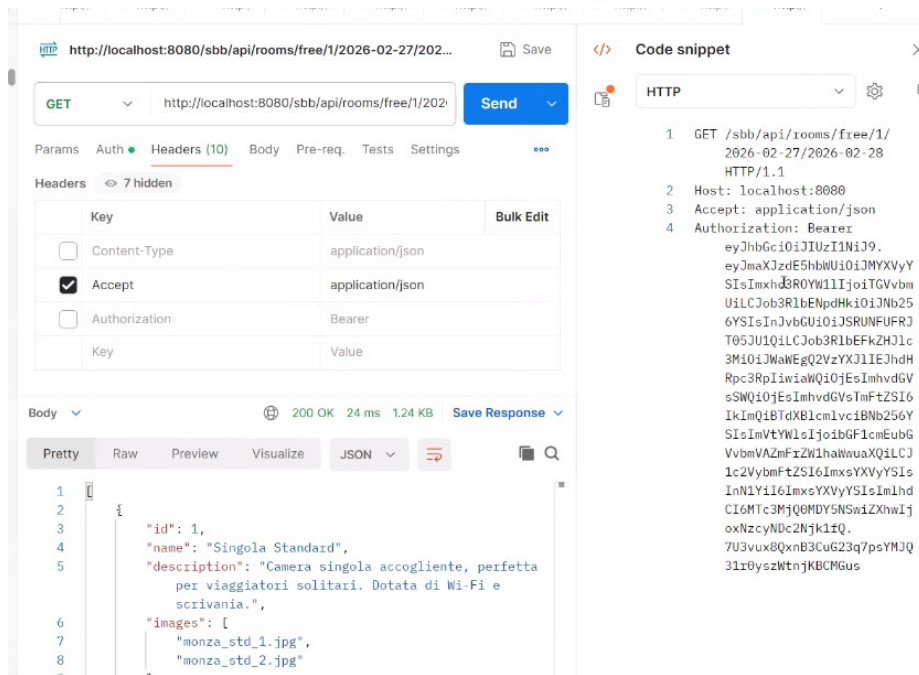
Quel token,



apparentemente lungo e casuale, è in realtà una stringa codificata in **Base64** composta da tre parti separate da un punto: l'header (algoritmo di firma), il payload (dati dell'utente) e la firma digitale che garantisce l'integrità del token. Da questo momento il client dovrà allegare questo token nell'header **Authorization: Bearer <token>** per accedere alle risorse protette.



```
doTest():void{
  this
    .http
    .get<any>(<"http://localhost:8080/sbb/api/rooms/free/1/2026-02-27/2026-02-28">)
    .subscribe({
      next: (data) => {
        console.log(data);
      },
      error: (error) => {
        console.log(error);
      }
    });
}
```



La differenza rispetto a prima è la presenza dell'header `Authorization: Bearer <token>`, dove il token è esattamente il JWT ricevuto durante il login .

Il metodo `this.http.get<any>()` di Angular `HttpClient` restituisce un **Observable<any>**. Non restituisce direttamente i dati, ma un oggetto Observable che rappresenta una operazione asincrona che emetterà il risultato nel futuro.

Cos'è un Observable

Un Observable è un tipo proveniente dalla libreria **RxJS**, e funziona in modo simile a una "promessa" ma più potente: non esegue nulla finché qualcuno non si **sottoscrive** ad esso. Ecco perché nel codice che hai mostrato viene chiamato `.subscribe()` subito dopo il `.get()`.

```

3
4
5   doTest():void{
6       this
7       .http
8       .get<any>("http://localhost:8080/sbb/api/rooms/free/1/2026-02-27/2026-02-28")
9       .subscribe({
10          next: (data) => {
11              console.log(data);
12          },
13          error: (error) => {
14              console.log(error);
15          }
16      });
17
18
19
20

```

Il ruolo di `.subscribe()`

Una volta chiamato `.subscribe()`, si definiscono due callback:

- **next**: viene eseguita quando la risposta arriva con successo, ricevendo i dati come argomento (`data`)
- **error**: viene eseguita se la richiesta fallisce, ricevendo l'oggetto errore

Il tipo `<any>`

Nel codice mostrato il tipo generico usato è `<any>`, il che significa che TypeScript non impone nessun controllo sul tipo dei dati ricevuti. In un'applicazione più strutturata, al posto di `any` si definirebbe un'interfaccia TypeScript (ad esempio `Room[]`) per avere il controllo dei tipi e il supporto dell'IDE

Facendo un paragone diretto con la `fetch` nativa di JavaScript:

- **next** corrisponde al blocco `.then()`, ovvero ciò che viene eseguito quando la risposta arriva con successo e i dati sono disponibili
- **error** corrisponde al blocco `.catch()`, ovvero ciò che viene eseguito quando qualcosa va storto (errore di rete, 403, 500, ecc.)

L'autenticazione dipende dal client.

```
14
15
16 doTest():void{
17     this
18     .http
19     .get<any>([
20         "http://localhost:8080/sbb/api/rooms/free/1/2026-02-27/2026-02-28"
21     ])
22     .subscribe({
23         next: (data) => {
24             console.log(data);
25         },
26         error: (error) => {
27             console.log(error);
28         }
29     });
30
31
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE esbuild + - □ □ ... | ↺ ×

```
>S C:\Users\Ferdinando\Desktop\securitytest> ng serve
```

Per farlo funzionare devo specificare un header


```

14
15
16 doTest():void{
17     this
18     .http
19     .get<any>({
20         "http://localhost:8080/sbb/api/rooms/free/1/2026-02-27/2026-02-28",
21         {
22             headers:{ "Authorization": "Bearer eyJhbGciOiJIUzI1NiJ9.eyJmaXJzdE5hbWUiOiJMYXVyY"
23         }
24     })
25
26     .subscribe({
27         next: (data) => {
28             console.log(data);
29         },
30         error: (error) => {
31             console.log(error);
32         }
33     });

```

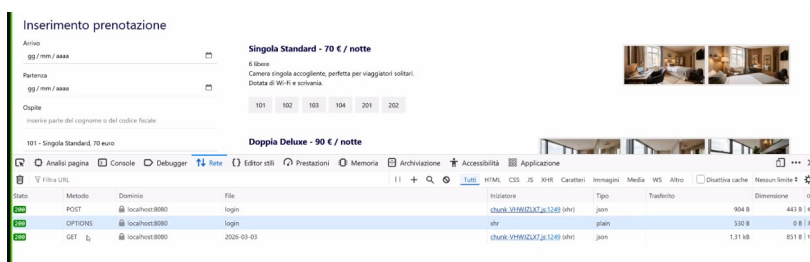
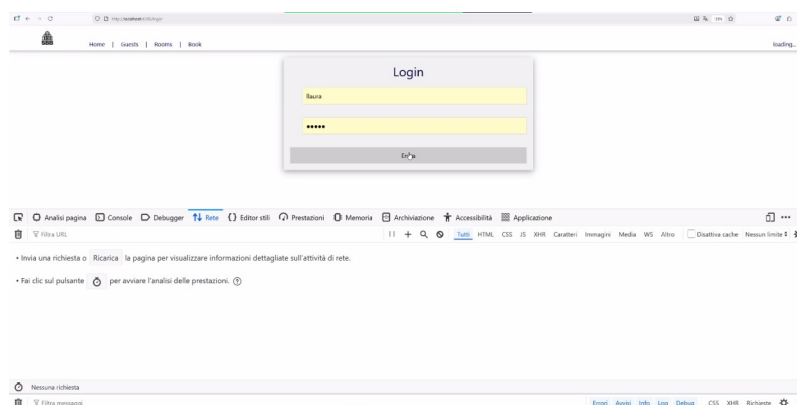
Questa versione aggiornata del codice mostra come passare il **token JWT nell'header della richiesta GET** direttamente da Angular. Rispetto alla versione precedente, il metodo `.get()` riceve ora un secondo argomento: un oggetto di configurazione con la proprietà `headers`, dove viene specificato `Authorization: "Bearer <token>"`.

Il flusso completo è quindi: login → ricezione del JWT → salvataggio del token → utilizzo del token nelle richieste successive tramite l'oggetto `headers`.

La comunicazione tra client e server nel mondo vero avviene nell'https. La password non la invio mai in chiaro. Il token viene sempre inviato sulla linea criptata.

C'è il rischio del *man in the middle* → ci sono una serie di programmi (wireshark) che possiamo usare per fare sicurezza, che analizzano il traffico di rete, e posso leggere il traffico che stai inviando se sono connesso alla stessa rete wifi.

Le connessioni di qualunque tipo DEVONO essere CRIPTATE.



La api da studiare.

```
14
73 |
74 |     /**
75 |         loginDTO => nome utente e password
76 |         mi arriva tramite post perché le chiamate get non sono adatte
77 |         i parametri sono visibili nell'indirizzo se passo le info come parametri
78 |     */
79 |     @PostMapping("/login")
80 |     public TokenDTO login(@RequestBody LoginDTO loginDTO) {
81 |         return service.login(loginDTO);
82 |     }
}
```

L'utilizzo dell'oggetto LoginDTO, che contiene le credenziali di accesso quali nome utente e password, avviene attraverso il metodo POST poiché le chiamate di tipo GET risultano inadatte a questo scopo. Tale scelta è determinata dal fatto che, trasmettendo le informazioni come parametri di una richiesta GET, i dati sensibili rimarrebbero visibili direttamente all'interno dell'indirizzo URL. .

```
5
6 |
7 |     public TokenDTO login(LoginDTO loginDTO) {
8 |         User user = userRepository.findByUsername(loginDTO.getUsername())
9 |             .orElseThrow(() -> new EntityNotFoundException(message: "User not found"));
10 |
11 |         if (!user.getPassword().equals(passwordHasher.toHash(loginDTO.getPassword()))) {
12 |             throw new RuntimeException(message: "Invalid password");
13 |         }
14 |
15 |         // il token viene inviato al client, il client ce lo rimanderà
16 |         return new TokenDTO(jwtService.generateToken(user));
17 |     }
}
```

Il processo di autenticazione prevede che, una volta verificate con successo le credenziali dell'utente, venga generato un token di sicurezza. Questo codice identificativo viene trasmesso dal server al client, il quale ha l'onere di memorizzarlo e di includerlo in ogni successiva richiesta inviata al sistema per attestare la propria identità e mantenere la sessione attiva.

Il metodo login riceve username e password dell'utente tramite un LoginDTO, cerca l'utente nel database per username lanciando una EntityNotFoundException (→ 403 Forbidden) se non esiste, confronta la password inserita con quella salvata hashando entrambe tramite passwordHasher.toHash() e lanciando RuntimeException se non corrispondono, e infine genera un token JWT tramite jwtService.generateToken(user) che restituisce al client incapsulato in un TokenDTO, il quale verrà usato per le autenticazioni successive senza reinviare le credenziali.

Il token deve contenere dei dati sulla sua generazione (frega niente) vengono messi in automatico. Poi un payload(il contenuto), che deve contenere i dati dell'utente e i suoi CLAIMS (le sue informazioni, i suoi diritti), e alla fine contiene una firma.

```
@Service
public class JwtService {

    // Chiave segreta (in un'app reale va nelle variabili d'ambiente, non nel codice!)
    // Deve essere lunga almeno 256 bit per HS256
    private static final String SECRET = "404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970";
```

Ogni server avrà la sua chiave (cambiata automaticata o generata all'avvio del server).

I CLAIMS sono espressi nel generateToken.

```
public String generateToken(User user) {
    Map<String, Object> claims = new HashMap<>();
    claims.put(key: "id", user.getId());
    claims.put(key: "firstName", user.getFirstName());
    claims.put(key: "lastName", user.getLastName());
    claims.put(key: "username", user.getUsername());
    claims.put(key: "role", user.getRole());

    claims.put(key: "email", user.getEmail());
    if (user.getHotel() != null) {
        claims.put(key: "hotelId", user.getHotel().getId());
        claims.put(key: "hotelName", user.getHotel().getName());
        claims.put(key: "hotelAddress", user.getHotel().getAddress());
        claims.put(key: "hotelCity", user.getHotel().getCity());
    }
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(user.getUsername())
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10)) // Scade tra 10 ore
        .signWith(getSignKey(), SignatureAlgorithm.HS256)
        .compact();
}
```

Il processo di generazione del token di sicurezza si articola attraverso una struttura tecnica definita che garantisce l'integrità e la disponibilità delle informazioni dell'utente. Ciascun token include automaticamente i dati relativi alla propria creazione, seguiti da un payload che costituisce il contenuto informativo principale. All'interno di quest'ultimo sono inseriti i dati dell'utente e i suoi claims, ovvero l'insieme delle informazioni e dei diritti d'accesso che lo riguardano. Nello specifico, la definizione dei claims avviene all'interno del metodo `generateToken`, dove vengono mappati attributi quali l'identificativo univoco, il nome, il cognome, lo username, il ruolo e l'indirizzo email dell'utente. La validità temporale del token è configurata per una durata di dieci ore dal momento dell'emissione. Infine, il token viene sigillato mediante una firma digitale basata sull'algoritmo HS256, utilizzando una chiave segreta univoca gestita dal server, la quale può essere generata all'avvio o aggiornata automaticamente per garantire la massima sicurezza.

Il primo punto separa l'**Header** dal **Payload**. L'header contiene informazioni tecniche, come il tipo di token e l'algoritmo di firma utilizzato, ad esempio l'HS256.

Il secondo punto separa il **Payload** dalla **Firma**. Il payload rappresenta il corpo del messaggio e contiene i claims, ovvero i dati dell'utente come lo username, l'ID, i ruoli ed eventuali dettagli sulla struttura alberghiera associata.

La sezione finale, che segue il secondo punto, è la **Firma**. Questa viene generata combinando le prime due parti codificate con una chiave segreta (SECRET) conservata sul server.

I claims sono effettivamente rappresentati come una mappa, nello specifico una `HashMap<String, Object>`, all'interno della logica di generazione del token. Questa struttura

a coppie chiave-valore permette di inserire dinamicamente diverse tipologie di informazioni riguardanti l'utente.

All'interno di questa mappa vengono inseriti i dati anagrafici e tecnici, come l'identificativo, il nome, il cognome, lo username, il ruolo e l'indirizzo email. La flessibilità della mappa consente inoltre di aggiungere condizionalmente ulteriori set di dati; ad esempio, se l'utente è associato a una struttura, vengono inclusi nella mappa anche i dettagli relativi all'hotel, come il codice, il nome, l'indirizzo e la città.

Una volta popolata, questa mappa viene passata al costruttore del token tramite il metodo `setClaims`, diventando ufficialmente il corpo informativo (payload) che sarà poi codificato e protetto dalla firma digitale.

La scelta di utilizzare una struttura di tipo `Map<String, Object>` per la gestione dei claims risponde a precise necessità tecniche di flessibilità e standardizzazione del formato JWT.

L'utilizzo di una stringa come chiave è dettato dalla specifica del protocollo, che richiede nomi univoci per identificare ogni informazione inserita nel payload, come ad esempio "id", "role" o "email". La scelta di `Object` come valore è invece fondamentale per permettere alla mappa di ospitare dati di natura eterogenea: all'interno dello stesso token possono coesistere numeri interi per gli identificativi, stringhe per i nomi o lo username, e potenzialmente strutture più complesse o booleani.

Questa impostazione è particolarmente utile nella logica di generazione del token, poiché consente di aggiungere campi in modo condizionale. Ad esempio, la mappa può essere estesa dinamicamente per includere i dati relativi all'hotel solo se l'utente ne possiede uno, senza dover definire una struttura dati rigida o predefinita per ogni tipologia di utente. In fase di finalizzazione, la libreria `JwtS` prende questa mappa e la converte automaticamente nel formato JSON che costituirà la parte centrale del token, garantendo che ogni valore venga serializzato correttamente in base al suo tipo originale.

Il server possiede una chiave segreta (SECRET), nota esclusivamente a esso e mai condivisa con il client o con l'esterno. Questa chiave è l'elemento fondamentale per generare e verificare la firma.

Al momento della creazione del token, il server prende l'header e il payload e li combina insieme utilizzando la propria chiave segreta tramite l'algoritmo HS256. Il risultato di questa operazione crittografica è la firma, ovvero la terza parte del token dopo il secondo punto.

Ogni volta che il client invia il token per accedere a una risorsa, il server esegue nuovamente lo stesso calcolo: prende l'header e il payload ricevuti dal client e prova a ricalcolare la firma utilizzando la sua chiave segreta.

Se l'utente avesse modificato anche solo un carattere del payload (ad esempio cambiando il proprio ruolo da "user" ad "admin"), il ricalcolo della firma produrrebbe un risultato differente da quella presente nel token inviato. Il server, riscontrando questa discrepanza, capirebbe immediatamente che i dati sono stati alterati o inventati, rifiutando la richiesta poiché il token non è più integro.

1. Richiesta di autenticazione

Il client avvia il processo inviando una richiesta di tipo **POST** al server, contenente l'oggetto `LoginDTO` con le credenziali (nome utente e password). L'uso del metodo POST è fondamentale poiché, a differenza del metodo GET, evita che i dati sensibili siano visibili direttamente nell'indirizzo URL.

2. Validazione e generazione

Una volta ricevuti i dati, il server verifica l'esistenza dell'utente nel database tramite lo username. Successivamente, confronta la password ricevuta con quella archiviata dopo averla convertita in hash. Se le credenziali sono valide, il server procede alla creazione del token tramite il metodo `generateToken`.

3. Struttura e firma del token

Il server costruisce il payload del token utilizzando una mappa (`Map<String, Object>`) che contiene i **claims**, ovvero i dati e i diritti dell'utente (ID, ruolo, email ed eventuali dati dell'hotel). Per garantire che queste informazioni non vengano "inventate" o modificate, il server applica una firma digitale utilizzando l'algoritmo **HS256** e una **chiave segreta** (SECRET) nota solo al server stesso.

4. Trasmissione e memorizzazione

Il token firmato viene incapsulato in un oggetto `TokenDTO` e inviato come risposta al client. A questo punto, il client ha l'onere di salvare il token localmente. Da questo momento in poi, per ogni interazione successiva, il client dovrà restituire il token al server, il quale potrà verificarne l'integrità controllando che la firma corrisponda ancora al contenuto.

```
)
.subscribe({
  next: (json) => {
    console.log(json);
    const token = (json as any).token;
    // jwt = intestazione.corpo.firma
    const payload = JSON.parse(atob(token.split('.')[1]));
    console.log('Decoded Token Payload:', payload);
  },
  error: (error) => {
    console.log(error);
  }
});
```

```

    .subscribe({
      next: (json) => {
        console.log(json);
        const token = (json as any).token;
        // jwt = intestazione.corpo.firma
        let parts = token.split(".");
        let body = parts[1]; // c'è l'utente (subject), ci sono i suoi claims
        // traduce il codice che ci è arrivato dalla codifica che ci ha mandato a testo normale
        let translated = atob(body);
        console.log('Decoded Token Body:', translated);
        const payload = JSON.parse(atob(body));
        console.log('Decoded Token Payload:', payload);
      },
      error: (error) => {
        console.log(error);
      }
    });

```

Questo è codice TypeScript/Angular che gestisce la risposta dopo una chiamata HTTP di login, dentro un `.subscribe()`, che è il modo in cui Angular "ascolta" il risultato di una richiesta asincrona.

next: (json) => { Questa funzione viene eseguita quando la risposta arriva con successo. `json` è la risposta del server, che contiene il token JWT.

const token = (json as any).token; Estrae il token JWT dalla risposta. Il `as any` serve solo per dire a TypeScript di non fare controlli sul tipo in quel momento.

let parts = token.split("."); Un token JWT è composto da tre parti separate da un punto: `intestazione.corpo.firma`. Con `split(". ")` lo si spezza in un array di tre elementi.

let body = parts[1]; Si prende la seconda parte, cioè il **payload** (il corpo), che contiene le informazioni sull'utente: chi è, i suoi ruoli, la scadenza del token, ecc. È ancora codificato in Base64.

let translated = atob(body); `atob()` è una funzione nativa del browser che decodifica una stringa Base64 e la trasforma in testo leggibile. Quindi `translated` ora è una stringa JSON in chiaro.

const payload = JSON.parse(atob(body)); Converte quella stringa JSON in un oggetto JavaScript vero e proprio, così si possono leggere i singoli campi come `payload.sub`, `payload.role`, ecc.

error: (error) => { Questa funzione invece viene eseguita se la chiamata HTTP fallisce, e semplicemente stampa l'errore in console.

ASCII to Binary nel senso di decodifica Base64, cioè prende una stringa codificata in Base64 come `"SGVsbG8="` e la riconverte nella stringa originale `"Hello"`

atob → la funzione inversa è `btoa`, che prende una stringa normale e la codifica in Base64: `"Hello" → "SGVsbG8="`.

